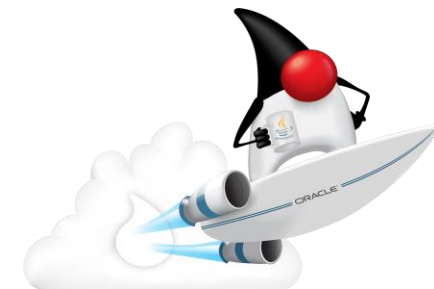


Programação Orientada a Objetos

- ✓ ESTRUTURAS DE DECISÃO
- ✓ MÉTODOS COM OBJETOS
- ✓ SOBRECARGA DE MÉTODOS



Estrutura de Decisão

As estruturas condicionais servem para impor uma condição para que algo seja executado

Condicional Simples

Executa as <instruções> se a condição for verdadeira



```
SE (condição) ENTÃO  
    <instruções>  
FIM SE
```



```
if (condição)  
{  
    instruções;  
}
```

Estrutura de Decisão

As estruturas condicionais servem para impor uma condição para que algo seja executado

- ✓ **if** $\Rightarrow$ seleciona um única ação ou um grupo de ações.

```
if(condição){  
    instruções;  
}
```

- ✓ **if/else** $\Rightarrow$ seleciona entre duas ações ou grupo de ações diferentes.

```
if (condição){  
    instruções 1;  
}  
else {  
    instruções 2;  
}
```

Estrutura de Decisão: Exemplo 1- Conta

Conta
numero: string saldo: double
Conta(n: String, s: double) imprimeDados(): void sacarValor(valor: double): void maiorSaldo(c: Conta): double

O valor só poderá ser retirado da conta caso exista saldo disponível, caso contrário mostre a mensagem “Saldo insuficiente”

Compara o saldo de duas contas e retorna o maior valor, se forem iguais, retornará o valor do objeto c

Estrutura de Decisão: Exemplo 2- Boletim

Boletim
n1: double n2: double media: double
Boletim(n1: double, n2: double) imprimeBoletim(): void calculaMedia(): void verificaConceito(): string

Calcula a média:
 $media = (n1 + n2) / 2$

MÉDIA	CONCEITO
8,0 ● ——— ● 10,0	A
6,0 ● ——— ○ 8,0	B
4,0 ● ——— ○ 6,0	C
Média abaixo de 4	D

Retorna o conceito,
conforme o valor da media,
de acordo com a tabela ao
lado

Observações importantes

- ✓ O comando **if** deve ter suas expressões contidas entre parênteses.
- ✓ O único argumento válido para um **if** é uma **expressão lógica ou variável booleana (condição)**.
- ✓ Preste atenção nos sinais de comparação (**=**) dentro de um **if**, pois eles podem ser confundidos com o operador de atribuição (**=**).
- ✓ As chaves não são obrigatórias para blocos **if** que têm **apenas uma instrução**, mas tome cuidado com erros, para evitar pequenos problemas na execução do código, utilize sempre as chaves.



Estrutura de Decisão Aninhada

```
if (condição1){  
    instruções 1  
}  
else{  
    if (condição2){  
        instruções 2  
    }  
    else{  
        instruções 3  
    }  
}
```

```
if (condição1){  
    if (condição2){  
        instruções1  
    }  
    else{  
        instruções2  
    }  
}  
else{  
    instruções3  
}
```

Estrutura switch-case

- ✔ É uma forma simples para se definir diversos desvios no código a partir de uma única variável.
- ✔ Usada quando se tem várias seleções com muitas alternativas.
- ✔ A partir da versão 7 permite teste com strings
- ✔ O comando switch testa somente condições simples



Estrutura switch-case



```
escolha (variável)
  caso valor1:
    Instruções1
  caso valor2:
    Instruções2
  caso valorn:
    InstruçõesN
  senão
    Instruções4
fim_escolha
```



```
switch (variável){
  case <valor1>:
    instruções 1;
    break;
  case <valor2>:
    instruções 2;
    break;
  case <valorn>:
    instruções n;
    break;
  default: instruções
    default;
}
```

Estrutura switch-case

Observações:

- ✔ todas as declarações case devem conter valores de um mesmo tipo;
- ✔ O tipo da variável deve ser compatível com os valores das declarações case.
- ✔ A declaração default é opcional.
- ✔ O break finalize o caso, retornando a execução após o comando switch. Case o comando break não seja inserido, todos os outros cases serão testados e executados.



Métodos recebendo e retornando objetos

Um método pode receber por parâmetro ou retornar qualquer tipo de informação, inclusive um objeto.

Produto
marca: string valor: float
cadastraDados(): void imprimeDados(): void calculaImposto(porcentagem: float): float duplicar(): Produto comparaCom(p: Produto): boolean



Métodos recebendo e retornando objetos

Um método pode receber por parâmetro ou retornar qualquer tipo de informação, inclusive um objeto.

Retorna um Produto

```
public Produto duplicar() {  
    Produto p = new Produto();  
    p.marca = this.marca;  
    p.valor = this.valor;  
    return p;  
}
```



Métodos recebendo e retornando objetos

Podemos somente receber Objetos e não retornar o mesmo tipo. Por exemplo, vamos criar um método de nome **comparaCom** que recebe outro Produto e retorna **true** se os valores deles forem iguais ou **false** se não forem.

```
public boolean comparaCom(Produto p){  
    return valor == p.valor;  
}
```

Recebe um Produto



Sobrecarga de Métodos

- ✓ Podemos criar vários métodos com o mesmo nome em uma classe, as ações são parecidas, mas os **parâmetros** necessários devem ser diferentes;
- ✓ **Sobrecarregar** um método é criar mais de um método com o **mesmo nome** mas **com parâmetros diferentes**;
- ✓ Os parâmetros podem se diferenciar em tipo e/ou em quantidade, de modo a alterar a **assinatura** do método.



Sobrecarga de Métodos

Assinaturas Diferentes

Assinatura do método é composta por:

- 1) Nome
- 2) Lista de parâmetros

```
public void print( int i ) { ... }  
public void print( float f ) { ... }  
public void print( String s ) { ... }  
public void print( String s, float g ) { ... }  
public void print ( String s, int i ) { ... }
```

```
public int print ( int i ) { ... }
```

É sobrecarga?

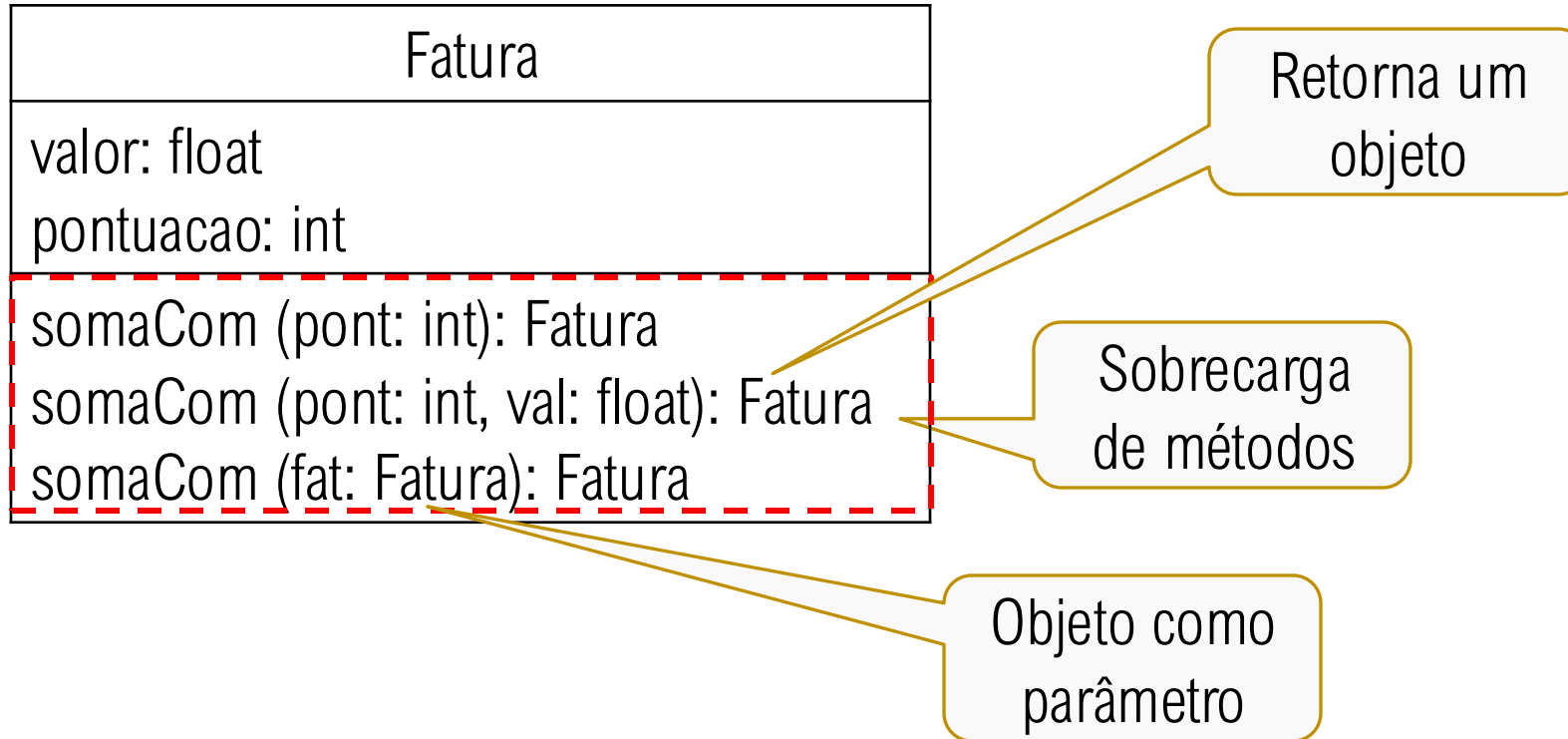


Sobrecarga de Métodos: Exemplo

```
public class Sobrecarga {  
    public int soma (int a) {  
        return a;  
    }  
  
    public int soma (int a, int b) {  
        return a+b;  
    }  
  
    public int soma (int a, int b, int c) {  
        return a+b+c;  
    }  
}
```



Sobrecarga de Métodos: Exemplo 3-Fatura



Sobrecarga de Métodos: Exemplo 3-Fatura

Método “somaCom”

Vamos criar um método que recebe um número inteiro e retorna uma nova fatura com a pontuação somada ao número que recebeu.

```
public Fatura somaCom(int pont) {  
    Fatura resp = new Fatura();  
    resp.pontuacao=pontuacao + pont;  
    return resp;  
}
```

Note que uma fatura nova é criada para poder ser carregada e retornada.

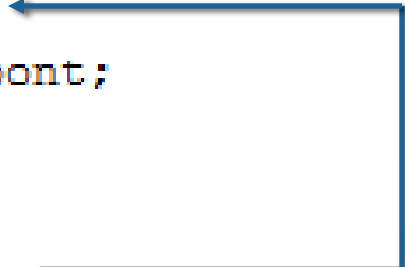


Sobrecarga de Métodos: Exemplo 3-Fatura

Método “somaCom”

Mas podemos querer que as somas sejam aplicadas tanto na pontuação quanto no valor. Vamos sobrecarregar novamente o método para receber dois números, um inteiro, e outro real.

```
public Fatura somaCom(int pont, float val) {  
    Fatura resp = new Fatura();  
    resp.pontuacao = pontuacao+pont;  
    resp.valor = valor + val;  
    return resp;  
}
```



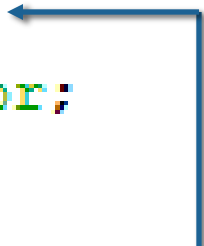
Note que uma fatura nova é criada para poder ser carregada e retornada.

Sobrecarga de Métodos: Exemplo 3-Fatura

Método “somaCom”

Agora vamos sobrecarregar o método somaCom de modo a receber como parâmetro outra fatura e retornar a soma delas.

```
public Fatura somaCom(Fatura fat) {  
    Fatura resp = new Fatura();  
    resp.valor = valor+ fat.valor;  
    return resp;  
}
```



Note que uma fatura nova é criada para poder ser carregada e retornada.

Sobrecarga de Métodos

Chamando Métodos

- ✔ Quando instanciamos o objeto, ele possui todos os métodos sobrecarregados. Como saber quando chamar um e quando chamar o outro se eles tem o mesmo nome?
- ✔ O próprio objeto vai saber qual método chamar dependendo dos parâmetros que você enviar para ele.



Sobrecarga de Métodos: Exemplo 3-Fatura

Chamando Métodos

Dependendo do
parâmetro

```
public class TesteFatura {  
  
    public static void main(String[] args) {  
        Fatura F1 = new Fatura();  
  
        F1.valor=100;  
        F1.pontuacao=60;  
  
        Fatura F2, F3, F4;  
  
        F2 = F1.somaCom(4);  
        F3 = F1.somaCom(F2);  
        F4 = F1.somaCom(3000, 6.530f);  
  
        F1.print();  
        F2.print();  
        F3.print();  
        F4.print();  
    }  
}
```

Exercícios de aplicação



Exercícios de aplicação

1- Considere os diagramas UML do seguinte item, observe que possui 2 construtores distintos. Crie a classe e depois, em uma classe Java principal, crie 2 objetos usando os construtores criados, também utilize os outros métodos.

Curso
nome: string quantidadealunos: int turma: string mensalidade: float
Curso() Curso(n:string, q:int, t:string, m:float) cadastraCurso(): void imprimeDados(): void calculaTotalMensalidade(): float



That's all Folks!